OR114-2 Midterm Project

Problem 1

1. Mathematical Formulation

Notation. Let C be the set of cars and K be the set of customer orders. Each car $c \in C$ has initial station h_c and level q_c. Each order $k \in K$ has requested level ℓ_k, pickup station p_k, return station r_k, pickup time s_k, return time e_k, and revenue R_k. Let t_{uv} be the moving time from station u to station v, and let B be the total relocation-time budget.

We build a directed acyclic route network. An initial arc $(c, k) \in A^0$ is created if car c can serve order k by level and can be moved from h_c to p_k in time. A transition arc $(c, i, j) \in A$ is created if the same car c can serve both orders i and j and can move from the return station of i to the pickup station of j after return handling and cleaning. More precisely, a transition from i to j is feasible only if

$$e_i + 240 + t_{r_i p_j} \leq s_j - 30.$$

The level rule is $q_c \in \{\ell_k, \ell_k + 1\}$ for every order k assigned to car c. Let $m_{ck} = t_{h_c p_k}$ for $(c, k) \in A^0$, and let $m_{ij} = t_{r_i p_j}$ for $(c, i, j) \in A$.

Parameter	
C	The set of cars
K	The set of customer orders
A^0	The set of feasible initial arcs from a car to its first order
A	The set of feasible order-to-order transition arcs for a car
h_c	The initial station of car c
q_c	The level of car c
p_k	The pickup station of order k
r_k	The return station of order k
s_k	The pickup time of order k
e_k	The return time of order k
ℓ_k	The requested level of order k
R_k	The revenue earned if order k is accepted
t_{uv}	The moving time from station u to station v
B	The total moving-time budget
m_{ck}	The moving time of initial arc $(c, k) \in A^0$
m_{ij}	The moving time of transition arc $(c, i, j) \in A$
Decision variable	
$x_{ck} = \begin{cases} 1, & \text{if car } c \text{ starts its route with order } k, \\ 0, & \text{otherwise,} \end{cases}$	$(c, k) \in A^0$.
$z_{cij} = \begin{cases} 1, & \text{if car } c \text{ serves order } j \text{ immediately after order } i, \\ 0, & \text{otherwise,} \end{cases}$	$(c, i, j) \in A$.
$y_k = \begin{cases} 1, & \text{if order } k \text{ is accepted,} \\ 0, & \text{otherwise,} \end{cases}$	$k \in K$.

Table 1: List of notations

With these decision variables, the IEDO planning problem can be formulated as the following integer program.

Variables associated with infeasible arcs are not created and are treated as zero in the expressions below.

$$
\begin{aligned}
\max \quad & \sum_{k \in K} R_k y_k - 2 \sum_{k \in K} R_k (1 - y_k) \\
\text{s.t.} \quad & \sum_{c:(c,k) \in A^0} x_{ck} + \sum_{(c,i,k) \in A} z_{cik} = y_k && \forall k \in K, \\
& \sum_{k:(c,k) \in A^0} x_{ck} \le 1 && \forall c \in C, \\
& \sum_{j:(c,i,j) \in A} z_{cij} \le x_{ci} + \sum_{h:(c,h,i) \in A} z_{chi} && \forall c \in C,\ i \in K, \\
& \sum_{(c,k) \in A^0} m_{ck} x_{ck} + \sum_{(c,i,j) \in A} m_{ij} z_{cij} \le B, \\
& x_{ck} \in \{0,1\} && \forall (c,k) \in A^0, \\
& z_{cij} \in \{0,1\} && \forall (c,i,j) \in A, \\
& y_k \in \{0,1\} && \forall k \in K.
\end{aligned}
$$

The first constraint says that every accepted order has exactly one predecessor, either an initial car-to-order arc or an order-to-order transition arc. The second constraint ensures that each car starts at most one route. The third constraint is the route-continuity constraint: a car can leave an order only if it has first entered that order. Since all transition arcs go forward in time, this also prevents cycles. The fourth constraint enforces the total employee moving-time budget. The objective maximizes total sales revenue minus rejection compensation; equivalently, for a fixed instance it maximizes accepted revenue.

2. Optimal Solution for Instance 05

The optimization model accepts 8 out of 10 orders, including Orders 3, 4, 5, 6, 7, 8, 9, and 10. Orders 1 and 2 are rejected. The accepted sales revenue is 117,000, and the final profit after rejection compensation costs is 106,800. The total relocation time used is 1,080 minutes, which satisfies the relocation budget limit ($1080 \le 1200$). Only one vehicle upgrade is required in the final solution, where a level-2 vehicle is assigned to Order 6, which requests a level-1 vehicle.

3. Operational Plan

Order	Car	Upgrade	Pickup → Return	Rental Period	Revenue
3	3	No	5 → 1	01/02 03:30 – 01/04 11:30	5,600
4	1	No	1 → 2	01/03 12:00 – 01/05 15:00	5,100
5	6	No	6 → 6	01/01 00:00 – 01/04 20:00	36,800
6	4	Yes	3 → 4	01/04 23:00 – 01/05 14:00	1,500
7	5	No	2 → 1	01/02 03:30 – 01/04 23:30	27,200
8	6	No	6 → 6	01/05 04:30 – 01/05 14:30	4,000
9	4	No	3 → 4	01/01 19:30 – 01/04 15:30	27,200
10	2	No	4 → 2	01/01 06:00 – 01/05 06:00	9,600

4. Vehicle Relocation Plan

Car	From	To	Depart	Arrive	Minutes
2	2	4	01/01 00:00	01/01 04:00	240
3	3	5	01/01 00:00	01/01 03:30	210
4	5	3	01/01 00:00	01/01 03:30	210
4	4	3	01/04 19:30	01/04 22:30	180
5	4	2	01/01 00:00	01/01 04:00	240

Problem 2

The required deliverable for this problem is the Python program. Our submitted function reads one instance file and returns the two required objects: `assignment`, a one-dimensional list indicating which car serves each order, and `relocation`, a two-dimensional list describing employee car movements. The implementation is written for the required Python interface and is designed to return a feasible plan within the three-minute limit. All design rationale, pseudocode, and complexity analysis are reported in Problem 3.

Problem 3

The hidden instances are graded under a three-minute time limit. This constraint is the main reason for our design. A full IP can make globally good decisions, but it is too expensive on large instances. A purely greedy method is fast, but it can waste scarce relocation minutes early. We therefore divide the algorithm into two parts:

- **Pre-build:** construct strong feasible solutions very quickly using two different approaches proposed by our team.
- **Optimization:** spend the remaining time on batch local IP repair, improving only small neighborhoods of the current solution.

In implementation, the pre-build stage is intended to finish quickly, and every local IP call in the optimization stage receives only a short time limit. The outer loop always checks the wall-clock deadline before starting another repair.

Pre-build Approach A: Revenue-first route insertion

Approach A is designed to be a fast and reliable high-profit route generator. It uses a single-pass greedy insertion process:

- **Order prioritization.** Orders are sorted primarily by decreasing revenue and secondarily by pickup time. The purpose is to give high-value orders early access to cars and relocation budget.
- **Chronological insertion search.** Each car maintains a chronological route. For an order k, the method scans compatible cars, i.e., exact level matches and valid one-level upgrades, and uses the pickup time to locate where k would be inserted in the route.
- **Feasibility evaluation.** The transition from the predecessor order to k and the transition from k to the successor order are checked. These tests enforce the one-hour possible late return, three-hour cleaning, thirty-minute readiness requirement, station moving time, level compatibility, and remaining global relocation budget.
- **Lexicographical selection.** Among all feasible insertions for the current order, the selected insertion minimizes marginal moving time first, upgrade penalty second, local idle time third, and car ID last.

If i and j are the predecessor and successor around k, the marginal moving time is

$$\Delta\text{move} = T_{r_i,p_k} + T_{r_k,p_j} - T_{r_i,p_j},$$

with the obvious modification when k is inserted as the first or last order of a car route. The exact key used by the selection rule is

$$(\Delta\text{move},\ \text{upgrade penalty},\ \text{local idle time},\ \text{car ID}).$$

Thus Approach A prioritizes revenue while still conserving relocation budget and avoiding unnecessary upgrades. For very small instances, an exact solver may be used as a fallback, but this step is bypassed for large hidden instances. Let $\bar{r}$ be the average route length. Sorting orders costs $O(n_K \log n_K)$. For each order, the algorithm may check n_C cars and inspect a local insertion position, so the overall complexity is approximately $O(n_K n_C \bar{r})$ in the worst case.

Pre-build Approach B: Demand-aware look-ahead

Approach B is intended to fix the main weakness of simple greedy assignment: it looks beyond the immediate order. The planning horizon is divided into six-hour buckets. For each station, car level, and bucket, we estimate future demand value. A car of level l contributes to the ability to serve level l and level $l-1$ orders, reflecting the free one-level upgrade rule. The table is computed by adding the value of each future pickup to its pickup station and requested level, then using a rolling four-bucket window, i.e., about 24 hours, to estimate how useful a car will be near a station in the near future.

When evaluating whether car c should serve order k, Approach B uses a score of the following form:

$$3R_k + w_f F(\text{return station}, q_{\text{after}}) - w_o F(\text{current station}, q_{\text{now}}) - w_m M(1 + 3B_{\text{used}}/B) - w_i I - w_u U.$$

Here M is the relocation time, I is idle time, and U is the upgrade penalty. The term $F(\cdot)$ is the future demand score. The positive future term rewards returning a car to a useful future location, while the opportunity cost penalizes removing a car from a station that will soon need it. The factor $3R_k$ appears because total profit can be rewritten as

$$\sum_{k\in A} R_k - 2\sum_{k\notin A} R_k = 3\sum_{k\in A} R_k - 2\sum_{k} R_k,$$

so accepting one more order improves the objective by $3R_k$ relative to rejecting it.

Approach B is then executed through a sequence of phases:

- **Initial greedy assignment.** Several parameter variants are tried, such as bucketed density order, chronological order, revenue-density order, and revenue-first order. Each variant greedily assigns orders to cars using the score above.
- **Shortage-bucket repair.** Rejected high-value orders are grouped by pickup station, time bucket, and requested level. Buckets with high remaining value and low expected inbound relocation effort are repaired first.
- **Limited route insertion.** High-value rejected orders are tested for direct insertion into existing car routes. Only nearby chronological positions and a capped number of candidate cars are checked.
- **One-removal swaps.** If direct insertion is impossible, the method tries replacing one lower-value accepted order in a compatible route with a higher-value rejected order.
- **Limited local replacement.** As a final lightweight repair, the method tries replacing the last order on a compatible car route, which is a cheap way to correct some greedy end-of-route mistakes.

Each proposed repaired route is simulated from the car's initial station and ready time, so timing, level compatibility, and relocation budget are checked again before the route replaces the old one. On very small instances, a bounded exact DFS is used as a final improvement; this is automatically skipped on large hidden instances.

If Q is the number of six-hour buckets, building the look-ahead table costs $O(n_K + n_S n_L Q)$. The greedy assignment scan is $O(n_K n_C)$ for each parameter variant. Repair loops are capped by constants in the implementation, so their cost is bounded for grading-time control. The pre-build stage runs both Approach A and Approach B and keeps the feasible solution with larger profit.

Optimization: batch local IP repair

The pre-build stage is fast, so we use the remaining time to improve the solution by exact optimization on small neighborhoods. First, the assignment is converted into routes for all cars. In each iteration, we sample a small batch of cars whose routes are relatively inefficient. The orders currently served by these cars are released, while routes of all other cars remain fixed. We then add a capped list of compatible high-revenue candidate orders, with priority given first to the released orders and then to currently rejected orders with large revenue. This keeps the local IP small while still giving it a chance to recover from a bad earlier assignment.

For this batch, we solve a local network-flow IP. It uses start variables for a selected car's first order, link variables for consecutive orders on a selected car, and acceptance variables for candidate orders. The local constraints are the same logical constraints as in Problem 1: each accepted order has one incoming arc, each selected car starts at most one route, flow is conserved, and the moving time used by the repaired batch does not exceed the remaining budget. If the local IP returns a set of routes with higher total profit and feasible moving time, we replace the old routes; otherwise, the incumbent solution is kept. Since the incumbent is never overwritten by a worse or infeasible repair, the algorithm can stop at any time and still return a feasible plan.

The full procedure is:

Input: instance file and three-minute time limit.
$A^{(1)} \leftarrow$ Approach A construction.
$A^{(2)} \leftarrow$ Approach B construction.
$A \leftarrow \arg\max\{\text{profit}(A^{(1)}), \text{profit}(A^{(2)})\}$.
while time remains do
  sample a batch of low-efficiency car routes;
  release those routes and build a candidate order set;
  solve the local IP with a short per-IP time limit;
  accept the repair only if profit improves;
return A and its relocation plan.

Suppose a local batch has b cars and at most m candidate orders. The local IP contains $O(bm)$ start arcs and $O(bm^2)$ order-to-order arcs in the worst case, so it has $O(bm^2)$ binary variables and constraints. In our implementation both b and m are capped, and each local IP has a short time limit τ. Therefore, under total time budget T, the number of expensive repair attempts is at most on the order of T/τ, plus the cheaper candidate selection and route-conversion overhead. Thus the algorithm uses exact optimization where it is most useful, while the total running time is controlled by the three-minute wall-clock limit.

Problem 4

To make our algorithm convincing beyond the five public instances, we generated random instances with our scenario generator. The planning horizon always starts at 2023/01/01 00:00. The fixed values are $n_S = 100$ stations and $n_L = 10$ levels. For every instance, n_D is sampled uniformly from 7 to 100. Hourly rates are generated as a strictly increasing sequence: the first base rate starts from a random value between 150 and 250, and each next level adds a random increment between 50 and 150. Pickup times are rounded to 30-minute slots and generated within the first $n_D - 2$ days so that rentals can finish inside the horizon; rental duration is uniformly sampled from 2 to 48 hours. Moving time is symmetric with $T_{ii} = 0$ and $T_{ij} = 90$ minutes for $i \neq j$.

The generator creates fourteen scenarios. Unless stated otherwise, $n_C = 100$. In high-load scenarios S2 and S4, $n_C = 80$; S2 uses $n_K = 10n_C = 800$ and S4 uses $n_K = 3n_C = 240$. In the other scenarios, n_K equals the listed load ratio times

n_C. Level distribution is uniform except S5, where orders are mostly low odd levels while cars are mostly high even levels. Station distribution is uniform except S6, where orders move from even pickup stations to odd return stations, and S7–S8, where returns are concentrated around one or two hub groups. Time distribution is uniform except S9, which weights weekend days, and S10–S11, which create one or three normal peak centers. Budgets are either 10^6, 0, $30n_D$, or $300n_D$.

Scenario	Main factor	Ratio $n_K : n_C$	Levels	Stations	Time/Budget
S1	Baseline	1:1	random	random	random, $B = 10^6$
S2	Very high load	10:1	random	random	random, $B = 10^6$
S3	Low load	1:3	random	random	random, $B = 10^6$
S4	High load	3:1	random	random	random, $B = 10^6$
S5	Level mismatch	1:1	8:2 skew	random	random, $B = 10^6$
S6	Forced relocation	1:1	random	odd/even	random, $B = 300n_D$
S7	One hub	1:1	random	hub 1G	random, $B = 300n_D$
S8	Two hubs	1:1	random	hub 2G	random, $B = 300n_D$
S9	Weekend effect	1:1	random	random	weekend, $B = 10^6$
S10	One peak	1:1	random	random	one peak, $B = 10^6$
S11	Three peaks	1:1	random	random	three peaks, $B = 10^6$
S12	No relocation budget	1:1	random	random	random, $B = 0$
S13	Tight budget	1:1	random	random	random, $B = 30n_D$
S14	Moderate budget	1:1	random	random	random, $B = 300n_D$

The first benchmark uses the generated data in `data/exp_instances`. It contains all 14 scenarios with 30 instances per scenario. Let A be the accepted reward of a solution and T be the total reward of all input orders. Then the project profit is

$$\text{profit} = A - 2(T - A) = 3A - 2T.$$

For a fixed instance, T is constant, so maximizing profit is equivalent to maximizing $3A$. Whenever we compare a heuristic against the IP optimum in the tables and histograms, we use the accepted-reward optimality gap. For any method $m \in \{\text{base}, \text{our}\}$,

$$\text{gap}_m = \frac{3A_{\text{IP}} - 3A_m}{3A_{\text{IP}}} \times 100\% = \frac{\text{profit}_{\text{IP}} - \text{profit}_m}{\text{profit}_{\text{IP}} + 2T} \times 100\%.$$

The benchmark keeps both references required by the project: the exact IP value as an upper benchmark and the chronological feasible heuristic as a lower benchmark. Therefore, each table reports the baseline gap to IP, our gap to IP, and the accepted-reward improvement over the baseline:

$$\frac{3A_{\text{our}} - 3A_{\text{base}}}{3A_{\text{base}}} \times 100\%,$$

together with the average order completion rate. To keep the tables readable, we report the 10-second version used for the larger 128-instance robustness study below.

Scenario	Base gap (%) $\mu \pm \sigma$	Our gap (%) $\mu \pm \sigma$	Improve (%) $\mu \pm \sigma$	Base CR (%) avg.	Our CR (%) avg.
S1	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	99.7	99.7
S2	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	99.3	99.3
S3	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	99.5	99.5
S4	0.19 ± 0.57	0.03 ± 0.19	0.16 ± 0.43	99.2	99.2
S5	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	99.7	99.7
S6	3.43 ± 10.15	0.00 ± 0.00	5.16 ± 15.87	95.8	95.8
S7	2.62 ± 7.09	0.00 ± 0.00	3.33 ± 9.15	96.7	97.2
S8	5.07 ± 11.30	0.05 ± 0.16	7.32 ± 17.99	93.9	94.8
S9	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	99.9	99.9
S10	7.79 ± 3.60	0.45 ± 0.50	8.11 ± 4.25	89.7	90.0
S11	0.18 ± 0.74	0.00 ± 0.00	0.18 ± 0.77	99.9	99.9
S12	4.64 ± 6.01	1.04 ± 2.86	4.16 ± 7.04	15.6	15.6
S13	38.61 ± 7.52	0.84 ± 1.36	63.86 ± 20.32	38.2	42.7
S14	4.20 ± 10.81	0.07 ± 0.28	6.21 ± 17.61	94.4	95.7

Table 3: All-scenario benchmark for the 10-second proposed method. Each row summarizes 30 generated instances. Gap is measured against the exact IP upper benchmark; improvement is measured against the chronological baseline.

For the final detailed study, we selected S6, S8, S10, S13, and S14 because they stress spatial imbalance, hub returns, temporal peaks, and relocation-budget sensitivity. We generated 128 instances per selected scenario. The exact IP model from Problem 1 was solved for every generated instance, and the heuristic side used the 10-second version of the proposed method. Because these instances have $n_C = 100$ and $n_K = 100$, the IP is still solvable; its value is therefore the optimal profit and an upper benchmark for any heuristic solution.

We also used a simple feasible lower-bound heuristic. This baseline first sorts all orders by pickup time, breaking ties by order ID, and then processes this chronological list. For each order, it assigns the feasible compatible car requiring the least immediate moving time; if no car is feasible, the order is rejected. It has no look-ahead and no repair, so a strong proposed algorithm should clearly beat it.

For the histograms, lower is better, and zero means matching the IP optimum. The accepted-reward normalization is important in tight-budget cases. For example, in S13_study_079 the IP profit is $-3{,}708$, while the chronological baseline profit is $-1{,}359{,}024$. Using |IP profit| as the denominator would give $1{,}355{,}316/3{,}708 = 365.51$, or 36,551.13%, only because the optimal profit is close to zero. The total input reward is 1,865,376, so the accepted-reward denominator is $-3{,}708 + 2(1{,}865{,}376) = 3{,}727{,}044$, and the reported gap is 36.36%.

Scenario	Base gap (%) $\mu \pm \sigma$	Our gap (%) $\mu \pm \sigma$	Improve (%) $\mu \pm \sigma$	Base CR (%) avg.	Our CR (%) avg.
S6	6.90 ± 13.72	0.00 ± 0.00	10.72 ± 22.87	90.5	90.5
S8	6.24 ± 12.09	0.06 ± 0.21	8.96 ± 18.65	91.7	92.7
S10	8.45 ± 3.64	0.69 ± 0.65	8.65 ± 4.47	89.9	90.3
S13	38.13 ± 8.48	1.29 ± 2.00	62.45 ± 22.29	32.7	36.2
S14	6.21 ± 12.65	0.13 ± 0.42	9.12 ± 19.97	91.7	93.0

Table 4: Five selected 128-instance scenario studies using the 10-second proposed method. Gap is measured against the exact IP upper benchmark; improvement is measured against the chronological baseline.

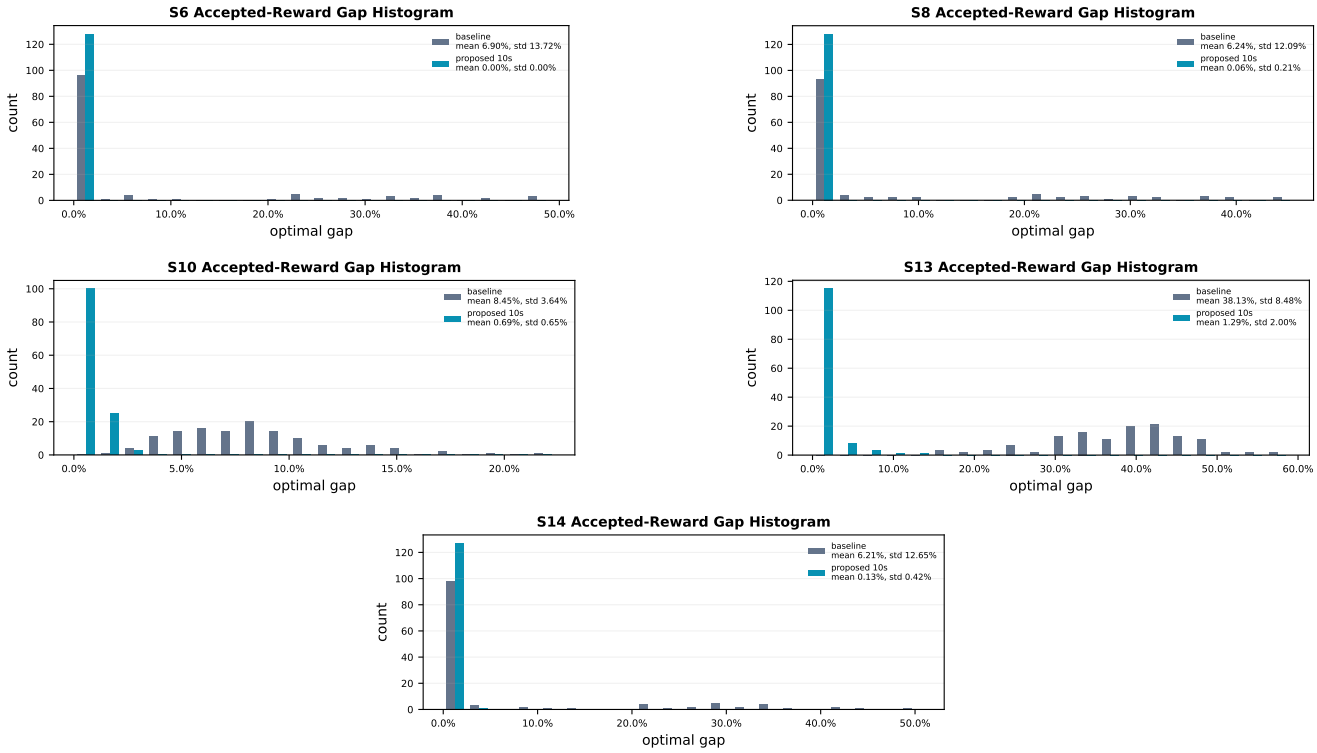

Figure 1: Optimality-gap histograms for the five selected scenarios. Each individual scenario figure reports the mean and standard deviation of the chronological baseline and our proposed algorithm.

The results support the algorithm design. In S6, the proposed algorithm matched the IP optimum on all 128 instances. In S8, S10, S13, and S14, its average gap stayed below 1.3%, while the chronological baseline ranged from 6.21% to 38.13%. S13 is still the hardest case because the relocation budget is very tight; the baseline loses many high-value assignments under this constraint, whereas the proposed optimization stage recovers most of the accepted reward. The experiment therefore shows both sides requested in the project statement: our algorithm is close to the exact optimum on solvable benchmark instances, and it is much better than a simple feasible heuristic. The mean and standard deviation of the improvement over the baseline are summarized in the table, the optimality-gap distributions are shown in the histogram figure, and the full set of instances, optimal plans, run-time records, and benchmark rows is preserved in `scenario_study_128_results.tgz` for reproducibility.